

Lab 4: VibeCoding — LLM Chat History

Project: Scenario 14 — 2048 (terminal game), repo SETAPESU26/14_2048. Tool used: Claude (claude.ai).

About this document. Student messages are reproduced exactly as sent. Claude's replies are condensed to their key points because the full replies contained long code listings; the final code is included in the appendix. For the platform's own full export, use the chat's share link (see README).

Message 1 — Student

[Uploaded: Lab_4_VibeCoding_Student_handout.pdf] https://github.com/SETAPESU26/14_2048

Claude's response (condensed)

- Read the lab handout and fetched the repo (README, board.py, game.py).
- Found four defects: (1) `result[-1] *= 2` lets a merged tile merge again, so `[2,2,4,0]` becomes `[8,0,0,0]`; (2) merges never add to the score; (3) `key not in "wasd"` is a substring test, so "" and "wa" pass validation; (4) undo is a stub and there is no move feedback.
- Gave a workflow with one commit per task, an example prompt for the LLM, a reference solution and unit tests.

Message 2 — Student

look at readme files and tell me what exactly i have to do and also i need to screen record tell me when i have to do

Claude's response (condensed)

- Unzipped the uploaded repo and confirmed it matched the GitHub version (`main.py` just calls `Game().run()`).
- Listed the four tasks (merge fix, game-state handling, undo and best score, move feedback), the required tests and what to submit (two videos, code, chat history).
- Recording plan: record the **before** video before changing any code, using a hardcoded `[2,2,4,0]` row; do all coding without recording; record the **after** video once all tasks are committed.

Message 3 — Student

can you tell me where to hard code

Claude's response (condensed)

- Pointed to `Board.__init__` in `board.py`, right after the two `add_random_tile()` calls, with a clearly marked temporary demo board.
- Gave the expected before and after results when pressing a, and warned to remove the block before committing.

Message 4 — Student

now i have the video record of the bug and now do all tasks mentioned

Claude's response (condensed)

- Implemented all four tasks in a copy of the repo with one git commit each, plus a test commit, and zipped it.
- Wrote `test_2048.py` with 27 unit tests; all passed, and a piped end-to-end run behaved correctly.
- Listed remaining steps for the student: run it, record the after video, remove any demo board, push to their own repo.

Message 5 — Student

can you first do it for task 1 what changes so i can add and commit

Claude's response (condensed)

- Task 1 changes only `board.py`: `slide_line_scored` uses `i += 2` after a merge so a merged tile cannot merge again; merges add to the score; the four move methods share one `_move` helper.
- Gave the full file, a check command and the commit command (`git add board.py`, not `-A`).

Message 6 — Student

now do for task2

Claude's response (condensed)

- `board.py`: added `WIN_TILE = 2048` and `has_won()`.
- `game.py`: win and no-move endings, exact-match input validation through a `DIRECTIONS` dictionary, and a new tile only after a move that changed the board.
- Explained each change and how to test the win and no-move boards.

Message 7 — Student

make it for task 3 and also to confirm if bug is fixed give me one liner python commands to check

Claude's response (condensed)

- `board.py`: added `snapshot()` and `restore()`. `game.py`: `undo_state` replaces the unused `history`; `undo()` restores board and score, creates no tile and is one level only; best score is kept after undo.
- Gave and ran one-line python `-c` checks for the merge fix, unchanged moves, win and no-move detection, and undo.

Message 8 — Student

task 4

Claude's response (condensed)

- Added `Game.feedback()`, built from the move's `last_merges` and `last_gain`, and made `run()` print exactly one line per move.
- Unchanged moves print "Nothing moved" and never say "Moved".
- Provided the test file, one-line checks, and the two commit commands.

Message 9 — Student

now can you change the readme file explaining what all i did

Claude's response (condensed)

- Kept the original README and appended "My Implementation Notes": defects found, what changed in each task, commit history, testing and how to play.
- Reminded the student to rename their videos to match the README, paste the chat link and review the wording.

Message 10 — Student

also i need to put the chat history so make a chat history

Claude's response (condensed)

- Created this document.

Appendix — Final code

board.py

```
import random

SIZE = 4
WIN_TILE = 2048

class Board:
    def __init__(self):
        self.grid = [[0] * SIZE for _ in range(SIZE)]
        self.score = 0
        self.last_gain = 0      # points scored by the most recent move
        self.last_merges = 0    # merges made by the most recent move
        self.add_random_tile()
        self.add_random_tile()

    def add_random_tile(self):
        empty = [(r, c) for r in range(SIZE) for c in range(SIZE) if self.grid[r][c] == 0]
        if empty:
            r, c = random.choice(empty)
            self.grid[r][c] = 4 if random.random() < 0.1 else 2

    @staticmethod
    def slide_line_scored(line):
        """Slide a line toward index 0.

        Returns (new_line, points_gained, merge_count). Every original tile
        takes part in at most one merge: after two tiles merge we skip both
        (i += 2), so the new tile can never merge again in the same move.
        """
        values = [x for x in line if x]
        result, gained, merges = [], 0, 0
        i = 0
        while i < len(values):
            if i + 1 < len(values) and values[i] == values[i + 1]:
                merged = values[i] * 2
                result.append(merged)
                gained += merged
                merges += 1
                i += 2
            else:
                result.append(values[i])
                i += 1
        return result + [0] * (SIZE - len(result)), gained, merges

    @staticmethod
    def slide_line(line):
        return Board.slide_line_scored(line)[0]

    def _get_line(self, i, direction):
        """Line i, oriented so that sliding toward index 0 means 'move in direction'."""
        if direction == "left":
            return self.grid[i][:]
        if direction == "right":
            return self.grid[i][::-1]
        col = [self.grid[r][i] for r in range(SIZE)]
        return col if direction == "up" else col[::-1]

    def _set_line(self, i, direction, line):
        if direction == "left":
            self.grid[i] = line
        elif direction == "right":
            self.grid[i] = line[::-1]
        else:
```

```

        col = line if direction == "up" else line[::-1]
        for r in range(SIZE):
            self.grid[r][i] = col[r]

def _move(self, direction):
    changed, gain, merges = False, 0, 0
    for i in range(SIZE):
        old = self._get_line(i, direction)
        new, g, m = self.slide_line_scored(old)
        if new != old:
            changed = True
            self._set_line(i, direction, new)
            gain += g
            merges += m
    self.score += gain
    self.last_gain = gain if changed else 0
    self.last_merges = merges if changed else 0
    return changed

def move_left(self):
    return self._move("left")

def move_right(self):
    return self._move("right")

def move_up(self):
    return self._move("up")

def move_down(self):
    return self._move("down")

def has_won(self):
    return any(WIN_TILE in row for row in self.grid)

def can_move(self):
    if any(0 in row for row in self.grid):
        return True
    for r in range(SIZE):
        for c in range(SIZE):
            if c + 1 < SIZE and self.grid[r][c] == self.grid[r][c + 1]:
                return True
            if r + 1 < SIZE and self.grid[r][c] == self.grid[r + 1][c]:
                return True
    return False

def snapshot(self):
    """Copy of everything needed to restore this board (grid + score)."""
    return [row[:] for row in self.grid], self.score

def restore(self, snap):
    self.grid = [row[:] for row in snap[0]]
    self.score = snap[1]

```

game.py

```

from board import Board

DIRECTIONS = {"a": "left", "d": "right", "w": "up", "s": "down"}

class Game:
    def __init__(self):
        self.board = Board()
        self.best_score = 0
        self.undo_state = None    # one level only: state before the last successful move

    def display(self):
        print("\n" + "+-----+-----+-----+-----+")
        for row in self.board.grid:

```

```

        print("|" + "|".join(f"{x:^6}" if x else f"{' ':^6}" for x in row) + "|")
        print("+-----+-----+-----+-----+")
    print("Score:", self.board.score, " Best:", self.best_score)

def move(self, key):
    """Apply a move. Returns True only if the board changed.

    A new tile is added only after a successful (changing) move.
    """
    if key not in DIRECTIONS:
        return False
    before = self.board.snapshot()
    changed = getattr(self.board, f"move_{DIRECTIONS[key]}")()
    if changed:
        self.undo_state = before
        self.board.add_random_tile()
        self.best_score = max(self.best_score, self.board.score)
    return changed

def undo(self):
    """Revert the last successful move (board + score). No tile is created.

    Best score is kept: it is the best of the current run. Returns False
    if there is nothing to undo.
    """
    if self.undo_state is None:
        return False
    self.board.restore(self.undo_state)
    self.undo_state = None
    return True

def feedback(self, key):
    """One action-level message for one accepted move."""
    name = DIRECTIONS[key].capitalize()
    if self.board.last_merges:
        plural = "s" if self.board.last_merges > 1 else ""
        return f"Moved {name}: {self.board.last_merges} merge{plural}, +{self.board.last_gain} points."
    return f"Moved {name}."

def run(self):
    print("2048 - W/A/S/D to move, U to undo, Q to quit.")
    while True:
        self.display()
        if self.board.has_won():
            print("You reached 2048! You win!")
            return
        if not self.board.can_move():
            print("No legal moves remain. Game over.")
            return
        key = input("> ").strip().lower()
        if key == "q":
            return
        if key == "u":
            print("Undone." if self.undo() else "Nothing to undo.")
            continue
        if key not in DIRECTIONS:  # exact match (the old substring test let "" and "wa" through)
            print("Use W/A/S/D.")
            continue
        if self.move(key):
            print(self.feedback(key))
        else:
            print("Nothing moved - try another direction.")

```